

DAY-26 — Git & GitHub Basics

CyberNotes • Detailed Beginner Reference

Learning goal: Build a practical understanding of git & github basics, connect the concepts to cybersecurity, and practice them safely in a local lab.

Study rule: Understand the command or concept before copying it. Security tools are powerful because they interact with real systems; your practice environment should therefore be intentionally controlled.

1. Core Concepts

Core idea: Git & GitHub Basics is a building block for practical cybersecurity. Learn what each component does, how it behaves normally, and how to verify your observations.

Why it matters: Security work depends on understanding normal system and network behavior before looking for weaknesses. This topic connects directly to Linux, networking, web security, monitoring, and troubleshooting.

Vocabulary: repositories, commits, branches, remotes, pull, push, README, .gitignore. Keep these terms in your own glossary and write one-line definitions as you study.

Reference	What to remember
git init	create repository
git status	working tree state
git add	stage changes
git commit	save snapshot
git log	history
git branch	branches
git pull	retrieve remote changes
git push	publish commits

2. How the Pieces Fit Together

- 1. repositories:** Start by defining what it is, where it appears, what normal behavior looks like, and what evidence you can observe. Then connect it to one neighboring concept. Example: understanding ports becomes more useful when you connect ports to services, processes, firewall rules, and logs.
- 2. commits:** Start by defining what it is, where it appears, what normal behavior looks like, and what evidence you can observe. Then connect it to one neighboring concept. Example: understanding ports becomes more useful when you connect ports to services, processes, firewall rules, and logs.
- 3. branches:** Start by defining what it is, where it appears, what normal behavior looks like, and what evidence you can observe. Then connect it to one neighboring concept. Example: understanding ports becomes more useful when you connect ports to services, processes, firewall rules, and logs.
- 4. remotes:** Start by defining what it is, where it appears, what normal behavior looks like, and what evidence you can observe. Then connect it to one neighboring concept. Example: understanding ports becomes more useful when you connect ports to services, processes, firewall rules, and logs.
- 5. pull:** Start by defining what it is, where it appears, what normal behavior looks like, and what evidence you can observe. Then connect it to one neighboring concept. Example: understanding ports becomes more useful when you connect ports to services, processes, firewall rules, and logs.
- 6. push:** Start by defining what it is, where it appears, what normal behavior looks like, and what evidence you can observe. Then connect it to one neighboring concept. Example: understanding ports becomes more useful when you connect ports to services, processes, firewall rules, and logs.
- 7. README:** Start by defining what it is, where it appears, what normal behavior looks like, and what evidence you can observe. Then connect it to one neighboring concept. Example: understanding ports becomes more useful when you connect ports to services, processes, firewall rules, and logs.
- 8. .gitignore:** Start by defining what it is, where it appears, what normal behavior looks like, and what evidence you can observe. Then connect it to one neighboring concept. Example: understanding ports becomes more useful when you connect ports to services, processes, firewall rules, and logs.
- 9. safe publishing:** Start by defining what it is, where it appears, what normal behavior looks like, and what evidence you can observe. Then connect it to one neighboring concept. Example: understanding ports becomes more useful when you connect ports to services, processes, firewall rules, and logs.

A useful mental model is **observe** → **understand** → **test** → **record** → **improve**. Avoid treating a tool's output as a conclusion by itself; the output is evidence that needs context.

3. Safe Practical Lab

- Step 1.** Define the lab target and the exact question you are trying to answer.
- Step 2.** Run the smallest safe observation or command that can answer that question.
- Step 3.** Read the output carefully; record important fields, errors, timestamps, and assumptions.
- Step 4.** Change one variable at a time and compare the result.
- Step 5.** Write a short conclusion and clean up temporary files or services.

Example lab sequence:

```
git init
git status
git add .
git commit -m 'Initial notes'
git log --oneline
```

4. Common Mistakes & Troubleshooting

- Running a command without checking its target, current directory, or privileges.
- Confusing a tool result with proof of a vulnerability or compromise.
- Ignoring error messages instead of reading the exact reason for failure.
- Changing several settings at once, making it difficult to identify what fixed or broke the lab.
- Saving sensitive real credentials, tokens, or personal data in practice notes or public repositories.
- Testing a public target simply because it is reachable; reachability is not authorization.

Troubleshooting checklist

- What system am I on?
- What exact command/request did I run?
- What did I expect?
- What happened instead?
- What does the error message say?
- Can I reproduce it in a clean local lab?
- What changed since the last successful attempt?

5. Deep-Dive Questions

Q1. What problem does repositories solve?

Notes: _____

Q2. What is the difference between normal behavior and a suspicious observation in this topic?

Notes: _____

Q3. Which command, log, packet, file, or response would you inspect first and why?

Notes: _____

Q4. What could cause a misleading result or false positive?

Notes: _____

Q5. How would you document the observation so another learner could reproduce it safely?

Notes: _____

Q6. Which security control reduces the risk associated with this topic?

Notes: _____

6. Revision Checklist

- I can explain git & github basics in my own words.
- I can identify the main terms without looking at the notes.
- I can perform the example in a controlled lab.
- I can interpret the output instead of only copying it.
- I know at least one defensive or troubleshooting use for the concept.
- I can document the result clearly and safely.

7. Safety & Scope

- Use these commands and tools only on systems you own, a local virtual machine, a CTF, an intentionally vulnerable application, or a target for which you have explicit permission.

- Keep a written scope: target, allowed actions, time window, and stop conditions. Do not scan or test random public systems.
- When practicing, save notes and outputs so you can explain what you observed rather than blindly running commands.

Next step: After Day-26, review your notes and connect this topic to the next subject in the series. The goal is not to memorize commands; it is to understand systems well enough to reason about security.

8. Quick Reference & Personal Notes

Topic: Day-26 — Git & GitHub Basics

Three things I learned

1. _____
1. _____
1. _____

Commands / concepts to memorize

- **git init** — create repository
- **git status** — working tree state
- **git add** — stage changes
- **git commit** — save snapshot
- **git log** — history
- **git branch** — branches
- **git pull** — retrieve remote changes
- **git push** — publish commits

My lab result

What I tested: _____

What I observed: _____

What I would investigate next: _____

Final reminder: A strong cybersecurity learner can explain both the offensive concept and the defensive reason for learning it. Keep practice authorized, reproducible, and documented.